
UNIVERSIDAD CENTRAL DEL ECUADOR
FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS
CARRERA DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

MANUAL DE EJECUCIÓN DEL PROGRAMA
FUENTE

SISTEMA WEB PARA LA EVALUACIÓN Y CLASIFICACIÓN
AUTOMÁTICA DE HOJAS DE VIDA IMPLEMENTANDO
INTELIGENCIA ARTIFICIAL

Meybili Tatiana Olivares Intriago
Lesly Salomé Salas Cueva

Quito - Ecuador
2026

índice

1.	INTRODUCCIÓN A LA EJECUCIÓN DEL PROGRAMA FUENTE.....	4
	1.1.Bienvenido	4
	1.2.Objetivos	4
	1.3.Acerca de la Documentación	4
	1.4.Estructura del programa fuente	4
2.	REQUISITOS PREVIOS PARA LA EJECUCIÓN	5
	2.1.Variables de entorno	5
3.	EJECUCIÓN DEL BACKEND	5
	3.1.Preparación del entorno e instalación de dependencias	5
	3.2.Configuración de las variables de entorno del backend	6
	3.3.Inicio del servicio backend	7
4.	EJECUCIÓN DEL FRONTEND	8
	4.1.Acceso al directorio y configuración de variables de entorno	8
	4.2.Instalación de dependencias	9
	4.3.Inicio del servidor de desarrollo del frontend	9
	4.4.Acceso a la aplicación	9
5.	ORDEN DE EJECUCIÓN DEL SISTEMA	10
6.	FINALIZACIÓN DE LA EJECUCIÓN	10
7.	POSIBLES PROBLEMAS DURANTE LA EJECUCIÓN	11

Lista de tablas

Tabla 1. Componentes principales del programa fuente	5
Tabla 2. Posibles problemas y soluciones durante la ejecución del sistema	11

Lista de figuras

Figura 1. Abrir Visual Studio Code desde GitHub Desktop.....	6
Figura 2. Ejecución correcta del backend en entorno local	8
Figura 3. Abrir Visual Studio Code desde GitHub Desktop para Frontend	8
Figura 4. Ejecución correcta del frontend en entorno local	9
Figura 5. Sistema web ejecutado correctamente.....	10

1. INTRODUCCIÓN A LA EJECUCIÓN DEL PROGRAMA FUENTE

1.1. Bienvenido

El presente manual describe el procedimiento requerido para ejecutar el programa fuente del Sistema Web para la Evaluación y Clasificación Automática de Hojas de Vida Implementando Inteligencia Artificial. La solución se encuentra compuesta por dos aplicaciones principales: el backend, implementado en Python y Flask, encargado del procesamiento de las hojas de vida, la clasificación y la comunicación con los servicios de almacenamiento; y el frontend, desarrollado con Next.js, React y TypeScript, encargado de proporcionar la interfaz web utilizada por el usuario. Para el funcionamiento correcto del sistema, el backend debe iniciarse primero y posteriormente debe ejecutarse el frontend, debido a que este último realiza solicitudes a los servicios expuestos por el backend.

1.2. Objetivos

El objetivo de este manual es orientar al usuario en la ejecución local del código fuente del sistema, indicando el orden de inicio de sus componentes, la preparación de los entornos, la instalación de dependencias y la configuración de las variables de entorno necesarias.

- Indicar el orden correcto para ejecutar los componentes del sistema.
- Describir la preparación y activación del entorno virtual del backend.
- Explicar la configuración de las variables de entorno.
- Describir la instalación de las dependencias del backend y frontend.
- Indicar los comandos necesarios para iniciar ambos componentes.
- Verificar que la aplicación se encuentre disponible correctamente.

1.3. Acerca de la Documentación

Este manual forma parte de la documentación técnica correspondiente al trabajo de titulación denominado “Desarrollo de un Sistema Web para la Evaluación y Clasificación Automática de Hojas de Vida Implementando Inteligencia Artificial”. Las instrucciones descritas presuponen que las herramientas requeridas para el proyecto han sido previamente instaladas según lo establecido en el ***Manual de instalación del software utilizado***. El manual fue elaborado por Meybili Tatiana Olivares Intriago y Lesly Salomé Salas Cueva, a quienes corresponden los derechos de autor del presente documento.

1.4. Estructura del programa fuente

El programa fuente se encuentra dividido principalmente en dos componentes: backend y frontend. La Tabla 1 presenta la función principal de cada componente dentro de la solución.

Componente	Descripción
Backend	Aplicación implementada en Python y Flask encargada del procesamiento, acceso a datos y servicios utilizados por el frontend.
Frontend	Aplicación web desarrollada con Next.js, React y TypeScript que proporciona la interfaz gráfica del sistema.

Tabla 1. Componentes principales del programa fuente

El orden de ejecución es: Backend -> Frontend -> Acceso desde el navegador.

2. REQUISITOS PREVIOS PARA LA EJECUCIÓN

Antes de iniciar el programa fuente, se debe comprobar que las herramientas detalladas en el *Manual de instalación del software* utilizado se encuentren disponibles en el equipo.

Adicionalmente, se debe disponer de:

- Código fuente del backend.
- Código fuente del frontend.
- Variables de entorno requeridas por ambos componentes.
- Conexión a Internet para acceder a la base de datos.

2.1. Variables de entorno

El sistema requiere archivos de configuración de variables de entorno independientes para el backend y el frontend:

- Backend: .env
- Frontend: .env.local

3. EJECUCIÓN DEL BACKEND

3.1. Preparación del entorno e instalación de dependencias

- El backend es el primer componente que debe ejecutarse. Para ello, desde GitHub Desktop, se debe clonar el repositorio denominado “clasificacion_hojas_vida” y abrir Visual Studio Code desde el proyecto clonado, como se indica en la Figura 1.

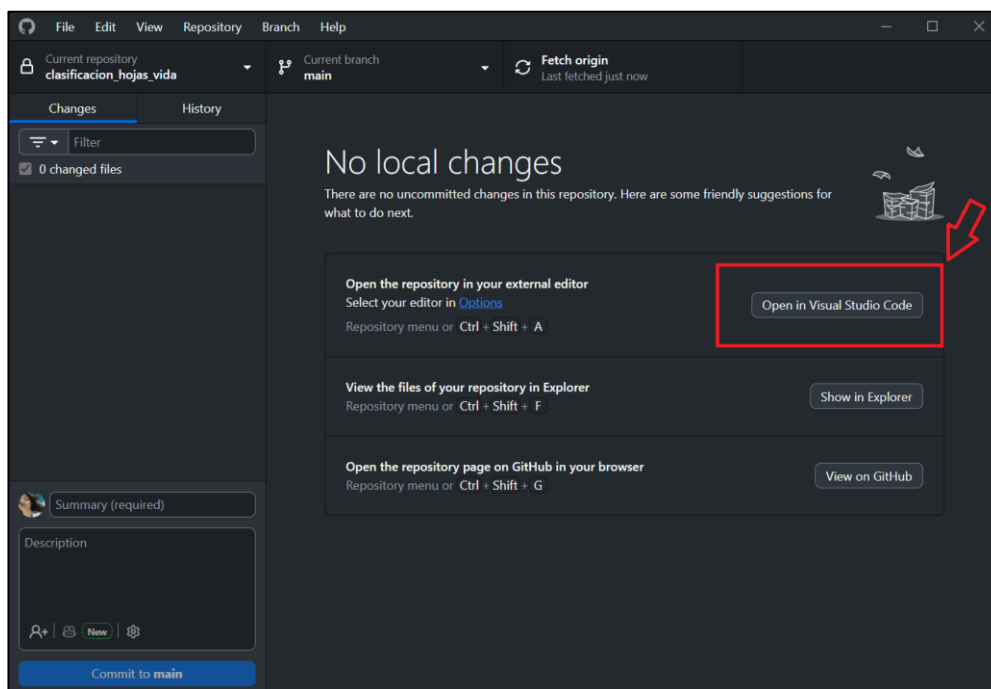


Figura 1. Apertura del proyecto backend en Visual Studio Code desde GitHub Desktop

- En una terminal de PowerShell, crear el entorno virtual de Python mediante el siguiente comando:

py -m venv .venv

- Posteriormente, activar el entorno virtual mediante el siguiente comando:

.\venv\Scripts\Activate.ps1

- Con el entorno virtual activo, instalar las bibliotecas requeridas por el backend, definidas en el archivo requirements.txt, mediante el siguiente comando:

pip install -r requirements.txt

3.2. Configuración de las variables de entorno del backend

Antes de iniciar el servicio, se debe crear el archivo **.env** en la raíz del proyecto. Este archivo contiene las variables necesarias para la conexión con Supabase, los límites de carga y los parámetros utilizados por los componentes de clasificación.

El archivo debe contener la siguiente configuración:

```
FLASK_ENV=development
HOST=127.0.0.1
PORT=5000
LOG_LEVEL=INFO
FRONTEND_URL=http://localhost:3000

SUPABASE_URL=https://btmzcrmykmdrlyreistk.supabase.co
SUPABASE_SECRET_KEY=<SOLICITAR_CONTENIDO>
SUPABASE_SERVICE_ROLE_KEY=<SOLICITAR_CONTENIDO>
```

```
SUPABASE_PROJECT_REF=btmzcrmykmdrlyreistk

MAX_CV_FILE_SIZE_MB=10
MAX_BATCH_FILES=50
MAX_REQUEST_SIZE_MB=30

MODEL_ARTIFACTS_DIR=artifacts
MODEL_ARTIFACT_BUCKET=model-artifacts
REPORTS_BUCKET=reports

TRAIN_TEST_SIZE=0.20
TRAINING_RANDOM_STATE=42
MIN_TRAINING_ROWS=10

TFIDF_MAX_FEATURES=5000
TFIDF_NGRAM_RANGE=1,2

KNN_DEFAULT_NEIGHBORS=3
TREE_MAX_DEPTH=10
TREE_MIN_SAMPLES_LEAF=3

AUTO_BUILD_DATASET_ON_TRAIN=True
DATASET_MIN_TEXT_LENGTH=80
DATASET_AUTO_LABEL_STRATEGY=relative
DATASET_POSITIVE_RATIO=0.45
DATASET_MATCH_THRESHOLD=0.55

CROSS_VALIDATION_ENABLED=True
CROSS_VALIDATION_FOLDS=5

AUTO_RETRAIN_ENABLED=True
AUTO_RETRAIN_NEW_PROFILES_THRESHOLD=1
AUTO_RETRAIN_MIN_THRESHOLD=1

INFERENCE_MODEL_CACHE_DIR=artifacts/inference_cache
TRAINING_REPORTS_DIR=artifacts/training_reports
```

Las variables `SUPABASE_SECRET_KEY` y `SUPABASE_SERVICE_ROLE_KEY` corresponden a credenciales privadas, por lo que sus valores reales no se incluyen en este manual.

3.3. Inicio del servicio backend

Una vez activado el entorno virtual, instaladas las dependencias y configurado el archivo `.env`, se debe iniciar el backend mediante el siguiente comando:

py run.py

Al ejecutarse correctamente, la consola debe mostrar que la aplicación Flask se ha iniciado en modo de desarrollo y que el servicio se encuentra disponible en la dirección local

<http://127.0.0.1:5000>, como se observa en la Figura 2.

```
(.venv) PS C:\Tesis\clasificacion_hojas_vida> py run.py
[2026-09-06 22:31:39,525] INFO in __init__: Application started in development mode
* Serving Flask app 'app'
* Debug mode: on
[2026-09-06 22:31:39,584] INFO in _internal: WARNING: This is a development server. Do
in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
[2026-09-06 22:31:39,584] INFO in _internal: Press CTRL+C to quit
[2026-09-06 22:31:39,585] INFO in _internal: * Restarting with stat
[2026-09-06 22:31:42,307] INFO in __init__: Application started in development mode
[2026-09-06 22:31:42,316] WARNING in _internal: * Debugger is active!
[2026-09-06 22:31:42,319] INFO in _internal: * Debugger PIN: 743-410-804
```

Figura 2. Ejecución correcta del backend en entorno local

Mientras el backend se encuentre en ejecución, esta terminal debe permanecer abierta para que el frontend pueda comunicarse con los servicios disponibles en el puerto 5000.

4. EJECUCIÓN DEL FRONTEND

4.1. Acceso al directorio y configuración de variables de entorno

- Para ejecutar el frontend, desde GitHub Desktop, se debe clonar el repositorio denominado “clasificacion_hojas_vida_frontend” y abrir Visual Studio Code desde el proyecto clonado, como se indica en la Figura 3.

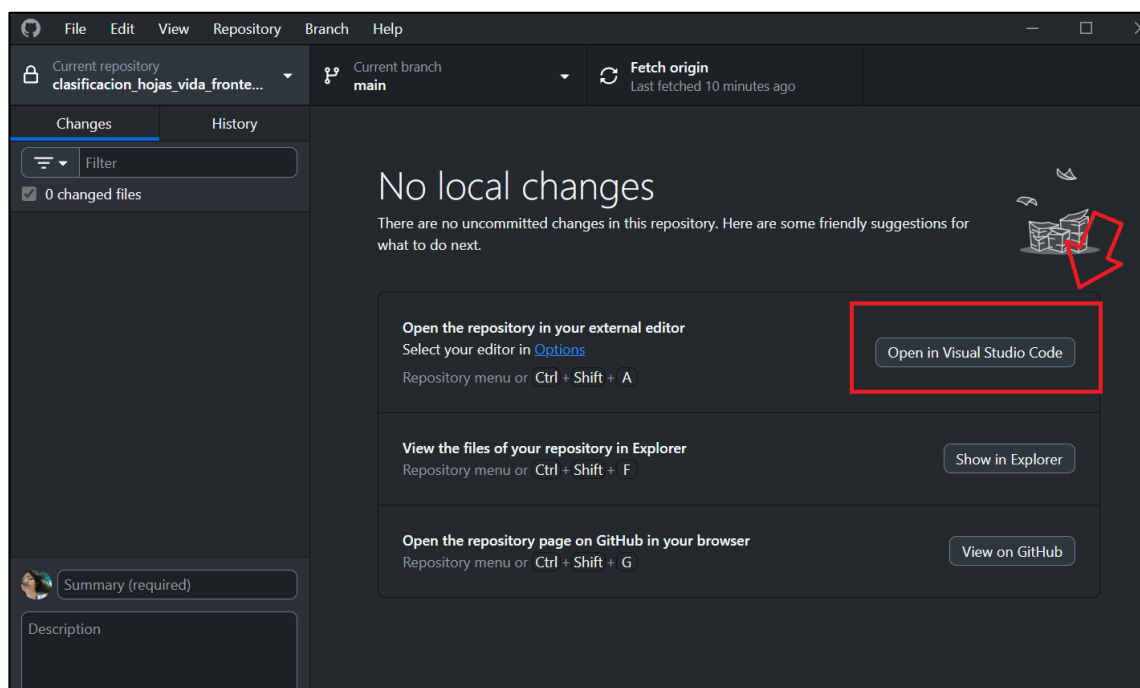


Figura 3. Apertura del proyecto frontend en Visual Studio Code desde GitHub Desktop

- Se debe crear el archivo **.env.local** en la raíz del proyecto. El archivo debe contener la siguiente configuración:


```
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:5000/api/v1
NEXT_PUBLIC_APP_NAME=Clasificador de Hojas de Vida
```

La variable `NEXT_PUBLIC_API_BASE_URL` establece la dirección del backend utilizado por la interfaz. Por esta razón, el backend debe encontrarse activo previamente en el puerto 5000.

4.2. Instalación de dependencias

Durante la primera ejecución del frontend, instalar las dependencias mediante:

yarn install

Si se presentan inconvenientes relacionados con el tiempo de espera de red, utilizar el siguiente comando:

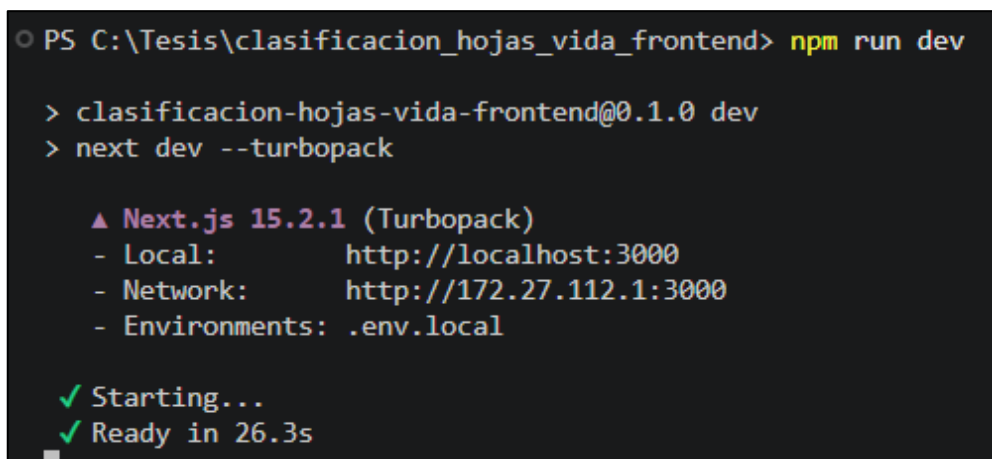
yarn install --network-timeout 600000

4.3. Inicio del servidor de desarrollo del frontend

Con las dependencias instaladas y el archivo `.env.local` configurado, se debe ejecutar el siguiente comando:

npm run dev

Al ejecutarse correctamente, la consola debe mostrar que la aplicación se ha iniciado y se encuentra disponible en la dirección local <http://localhost:3000>, como se observa en la Figura 4.



```
PS C:\Tesis\clasificacion_hojas_vida_frontend> npm run dev

> clasificacion-hojas-vida-frontend@0.1.0 dev
> next dev --turbo

  ▲ Next.js 15.2.1 (Turbopack)
  - Local:      http://localhost:3000
  - Network:    http://172.27.112.1:3000
  - Environments: .env.local

  ✓ Starting...
  ✓ Ready in 26.3s
```

Figura 4. Ejecución correcta del frontend en entorno local

4.4. Acceso a la aplicación

Una vez iniciados correctamente el backend y el frontend, se debe abrir un navegador web e ingresar a:

<http://localhost:3000>

La aplicación mostrará la pantalla principal del Clasificador de Hojas de Vida, como se observa en la Figura 5.

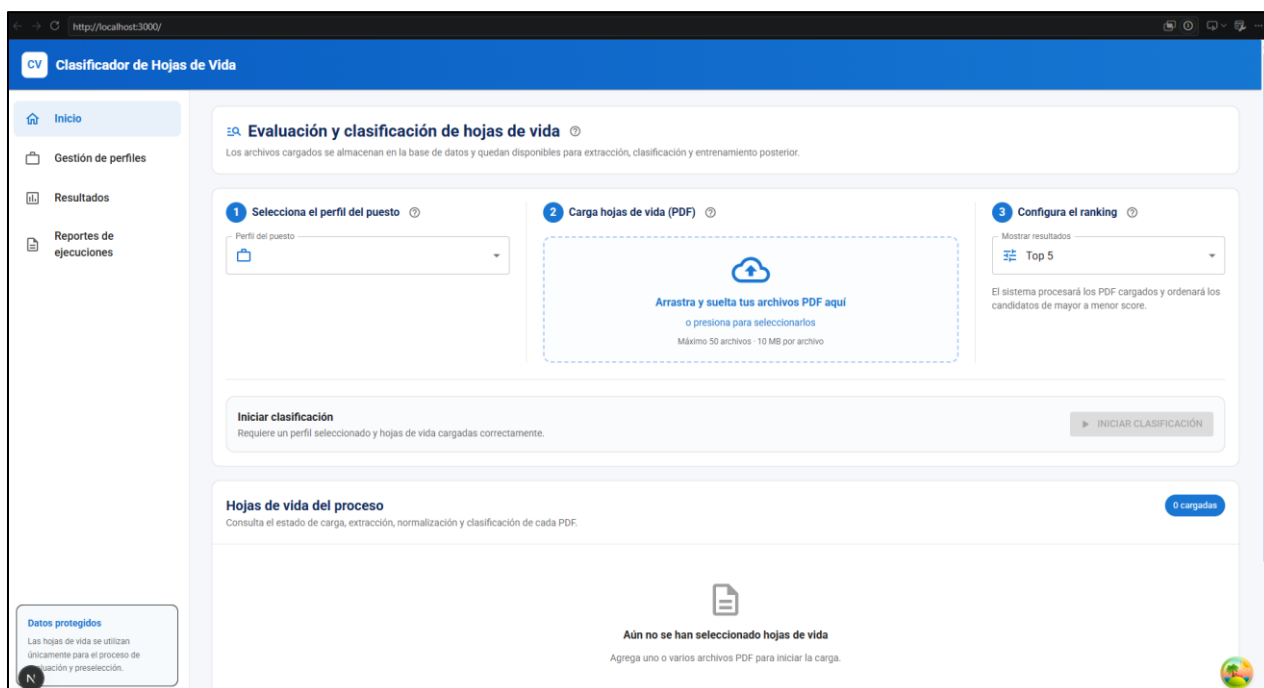


Figura 5. Sistema web ejecutado correctamente

5. ORDEN DE EJECUCIÓN DEL SISTEMA

Una vez realizada la configuración inicial del backend y frontend descrita en las secciones anteriores, para iniciar nuevamente el sistema se debe respetar el siguiente orden:

- Acceder al directorio del backend.
- Activar el entorno virtual .venv.
- Ejecutar el backend mediante py run.py.
- Mantener abierta la terminal del backend.
- Abrir una segunda terminal.
- Acceder al directorio del frontend.
- Ejecutar npm run dev.
- Acceder a <http://localhost:3000> desde el navegador.

Importante: El backend debe permanecer activo durante la utilización del frontend, ya que la interfaz realiza solicitudes a los servicios disponibles en <http://127.0.0.1:5000/api/v1>.

6. FINALIZACIÓN DE LA EJECUCIÓN

Para detener los servidores, se debe ingresar a cada una de las terminales en ejecución y presionar:

Ctrl + C

7. POSIBLES PROBLEMAS DURANTE LA EJECUCIÓN

Durante la ejecución del sistema pueden presentarse inconvenientes relacionados con la activación del entorno virtual, la instalación de dependencias, la configuración de las variables de entorno, la conexión entre los componentes o la disponibilidad de los puertos utilizados. En la Tabla 2 se presentan los problemas más frecuentes, sus posibles causas y las acciones recomendadas para solucionarlos.

Problema	Posible causa	Solución
El backend no inicia	El entorno virtual no se encuentra activo	Ejecutar <code>.\venv\Scripts\Activate.ps1</code> .
Se presentan errores de módulos en Python	Dependencias no instaladas	Ejecutar <code>pip install -r requirements.txt</code> .
El frontend no obtiene información	Backend detenido o dirección incorrecta	Verificar que <code>py run.py</code> permanezca activo y revisar <code>NEXT_PUBLIC_API_BASE_URL</code> .
<code>npm run dev</code> presenta errores de dependencias	Paquetes del frontend no instalados	Ejecutar <code>yarn install</code> .
El sistema no encuentra variables de entorno	Falta <code>.env</code> o <code>.env.local</code>	Crear o configurar el archivo correspondiente.
No existe conexión con Supabase	Credenciales faltantes o incorrectas	Verificar las variables de Supabase proporcionadas por el responsable técnico.
El puerto 3000 o 5000 está ocupado	Existe otra instancia del sistema en ejecución	Detener la instancia anterior y volver a iniciar el servicio.

Tabla 2. Posibles problemas y soluciones durante la ejecución del sistema